



AUDIT REPORT

September 2026

For



Table of Content

Executive Summary	04
Checked Vulnerabilities	05
Techniques and Methods	06
Types of Severity	07
Number of Security Issues per Severity	08
What We Tested and Verified: No Issues Found	09
Things We Could Not Verify Within This Scope	11
■ High Severity Issues	12
1. Seeded admin password gets silently reset on every restart, which undermines credential rotation	12
2. Confirmed live: Uniswap aggregator swaps can be redirected to an attacker's address by tampering routeData	13
3. LiFi bridge execution can be redirected to an attacker's address the same way, and more directly	15
■ Medium Severity Issues	17
1. Cost-basis endpoint lets anyone trigger a huge, unauthenticated RPC fan-out	17
■ Low Severity Issues	18
1. NestJS core has a moderate injection CVE (CWE-74, CVSS 6.1)	18
2. multer's high-severity DoS advisories exist but aren't reachable	19

3. No explicit HTTP body-size limit configured	20
4. No @MaxLength() on any string DTO field	21
5. Docker container runs as root	22
6. trust proxy hop-count assumption may not match the real network setup	23
7. axios has known advisories, needs a routine version bump	24
8. qs has known DoS advisories, reachable via public query parameters	25
■ Informational	26
Closing Summary & Disclaimer	30

Executive Summary

Project Name	Crackerswap
Project URL	https://crackerswap.com/swap
Overview	Crackerswap is a standalone NestJS DEX API. It supports Uniswap V3 on Base, Monad, Ink, and Robinhood Chain. The API is public, returns unsigned transactions, and requires no auth. Features • Swap quoting and unsigned-transaction building • Concentrated-liquidity position management (create, increase, decrease, collect fees, burn) • Pool list, pool info, top/new pools, by-token lookup • User LP positions with on-chain enrichment and optional historical cost basis • Analytics: pool graph, OHLCV candlestick, volume/price distribution, price-range graph, LP simulator
Audit Scope	The scope of this pentest was to analyze the Crackerswap Backend Source code review and Greybox testing for quality, security, and correctness. Source code: https://github.com/Crackerswap-Ecosystem/Dex-Backend URL: https://crackerswap.com/swap
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	3rd September 2026 - 11th September 2026
Updated Code Received	14th September 2026
Review 2	15th September 2026 - 16th September 2026
Fixed In	c6a9863424a37e725ca9232cae3c497ce4ba7e61

Checked Vulnerabilities

The code-review phase was split into 11 tracks, each closed out before moving to the next.

Track	Area
A	Secrets & config posture
B	Admin authentication & authorization
C	Transaction-building integrity (swap/liquidity/bridge calldata)
D	Chain/protocol routing correctness
E	Data layer / injection
F	External data trust (GeckoTerminal, CoinGecko, Uniswap API, LiFi)
G	Rate limiting / resource exhaustion / cost-abuse
H	Input validation & NestJS hardening
I	Dependency & supply chain
J	DeFi math correctness
K	Logging/observability hygiene

Techniques and Methods

Throughout the pentest of application, care was taken to ensure:

- Information gathering – Using OSINT tools information concerning the web architecture, information leakage, web service integration, and gathering other associated information related to web server & web services.
- Using Automated tools approach for Pentest like Nessus, Acunetix etc.
- Platform testing and configuration
- Error handling and data validation testing
- Encryption-related protection testing
- Client-side and business logic testing

Tools and Platforms used for Pentest

- ✓ Burp Suite
- ✓ DNSenum
- ✓ Dirbuster
- ✓ SQLMap
- ✓ Acunetix
- ✓ Neucli
- ✓ Nabbu
- ✓ Turbo Intruder
- ✓ Nmap
- ✓ Metasploit
- ✓ Horusec
- ✓ Postman
- ✓ Netcat
- ✓ Nessus and many more.

Types of Severity

Every issue in this report has been assigned with a severity level. There are four levels of severity, and each of them has been explained below.

■ High Severity Issues

A high severity issue or vulnerability means that your web app can be exploited. Issues on this level are critical to the web app's performance or functionality, and we recommend these issues be fixed before moving to a live environment.

■ Medium Severity Issues

The issues marked as medium severity usually arise because of errors and deficiencies in the web app code. Issues on this level could potentially bring problems, and they should still be fixed.

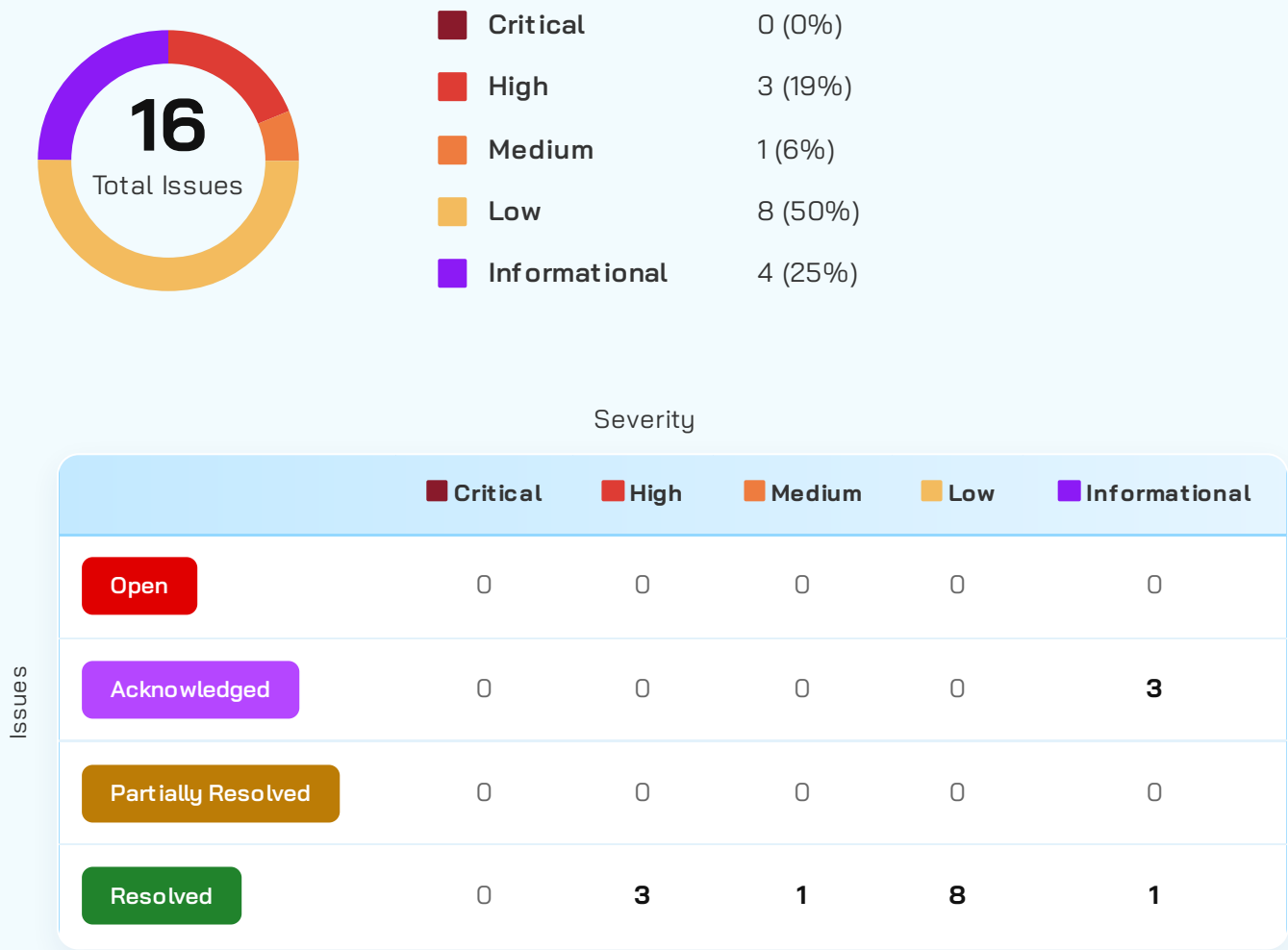
■ Low Severity Issues

Low-level severity issues can cause minor impact and or are just warnings that can remain unfixed for now. It would be better to fix these issues at some point in the future.

■ Informational

These are issues that indicate an improvement request, a general question, a cosmetic or documentation error, or a request for information. There is low-to-no impact.

Number of Issues per Severity



What We Tested and Verified: No Issues Found

This section is here so you can see the negative results too, not just the findings. Everything below was actively tested, not skipped.

Authentication and session management: The JWT strategy pins HS256 and checks issuer/audience explicitly, so it rejects alg: none tokens and tokens minted for other apps. It re-reads the admin from the database on every request, so a deactivated account or a revoked session can't outlive its access token. Refresh-token rotation correctly detects reuse of an already-rotated or revoked token and kills the whole session family, not just the one token. The CSRF double-submit check uses a constant-time comparison and is wired onto every cookie-authenticated mutation. Login has no user-enumeration leak: unknown-email and wrong-password give the identical error with near-identical timing, and logout only counts against real, active accounts. Role-based access control (admin vs super_admin) is correctly enforced on the two highest-value surfaces, admin-user CRUD and platform/bridge fee changes, and every other admin route needs at least a valid token. An admin can't deactivate or role-change themselves, can't delete themselves, and the seeded default admin can't be deleted, demoted, or deactivated through the API. Live-tested: every admin endpoint returns 401 with no token, and a single garbage-credential login attempt returns the generic error the source predicts.

Transaction-building integrity: Router, position-manager, factory, quoter, Universal Router, and Permit2 addresses are hardcoded per-chain constants for both Uniswap v3 and v4, never derived from request input. minOut/slippage for native (non-aggregator) swaps is always computed server-side from a fresh on-chain quote, never accepted from the client. Liquidity collect, decrease, and burn hardcode the payout recipient to the caller's own declared address, no separate override field exists for those calls. Platform fee percentages are clamped to the router's actual on-chain cap instead of silently overcharging or reverting unexpectedly.

Chain and protocol routing: Every protocol's deployment table is keyed exactly to the supported-chains list, so a request can never get a v3 call built against a v4-only chain or the reverse. Protocol dispatch uses a Map, not a plain object, so it's immune to prototype-pollution-style key collisions. Live-tested: a __proto__ field in a request body is inert, confirmed with a follow-up health check showing no process-wide side effect.

Injection and data layer: Zero use of Prisma's unsafe raw-SQL APIs anywhere in the codebase. Every raw query uses the safe, parameterized tagged-template form. Pagination is capped consistently across every public list/search endpoint we sampled (tokens, pools, screener, admin listings). Live-tested: a SQL-injection-shaped search string (' OR '1'='1) came back as a clean, empty result set with no error.

External data trust : No SSRF anywhere. Every outbound HTTP call target is built from a fixed, server-configured host, and request-derived values only ever populate query parameters, never a host or scheme. Every third-party API client (CoinGecko, GeckoTerminal, LiFi, Uniswap's Trading API) uses defensive fallback parsing that handles malformed or unexpected responses without crashing. API keys go out only via request headers to fixed hosts, never logged, never echoed back in responses.

Input validation: The global validation pipe rejects unrecognized request fields outright instead of just stripping them. Live-tested: malformed input comes back as a clean 400 with field-level messages, no stack trace, and an unexpected extra field gets rejected by name. No mass-assignment risk in any admin update path we checked. Every write explicitly names the fields it touches instead of spreading the request body directly.

CORS: Live-tested: a disallowed origin (evil-attacker.com) gets no Access-Control-Allow-Origin header at all, on either an admin or a public endpoint. The real frontend origin gets correctly echoed back, and Origin: null also gets nothing.

Logging: No credential material, passwords, JWTs, refresh tokens, API keys, shows up in any log line anywhere in the codebase. All logging goes through one consistent, structured logger, no raw console.* calls bypassing it anywhere.

DeFi math correctness: The fund-critical quote math, the part that directly feeds the on-chain minimum-output guarantee, is a faithful, verifiable port of Uniswap V3's own on-chain formulas, with a real safety fallback: it defers to the authoritative on-chain simulator whenever a trade could cross a liquidity boundary, instead of risking a wrong fast-path answer. Test coverage specifically targets the failure classes that actually matter here (negative-number rounding direction, boundary crossings, zero/negative clamping), not just happy paths.

Infrastructure: HTTP to HTTPS redirect confirmed live. Strong security headers (HSTS, CSP, frame-ancestors, etc.) present on both hosts. No sensitive files exposed on either host, .env, .git/*, docker-compose.yml, prisma/schema.prisma, and build artifacts all return clean 404s. The deliberate dependency pins (ws, ox) are still effective, neither appears in the current vulnerability scan. The Dockerfile hardcodes production mode at the image level, which independently confirms (alongside the live header/behavior checks) that production-only hardening, Swagger disabled, CORS wildcard rejected, dev secrets inactive, is genuinely active.

Things We Could Not Verify Within This Scope

This engagement covered the backend repository and the crackerswap.com/swap live site. A few things came up during the review that sit outside both of those: infrastructure details, other components, git history, and so on. Listed here so it's clear what's a limitation of scope rather than something we checked and found fine.

- The production bearer-token auth layer on `/v1/swap`, `/v1/liquidity/*`, and `/v1/bridge/execute`. Whatever issues and validates this token isn't in the backend repository or the `/swap` page, so we have no visibility into it. This is the reason M-1b is unverified rather than settled.
- Whether `ADMIN_PASSWORD` and `JWT_SECRET` were rotated as environment variables, not just changed in the database, in every environment that ever held the values leaked in earlier commits (SECURITY documents this leak; H-1 explains why a database-only change wouldn't actually fix it). This isn't observable from the repository or the live site, it's a fact about the deployment environment itself.
- Network topology in front of the GCP deployment. We can't confirm the real hop count reaching the Node process, which is what the trust proxy finding (L-6) hinges on. That's an infrastructure detail outside the repository and the live site.
- The live nginx configuration on the GCP VM. The repository only contains a commented template, not the deployed config, so we can't confirm the actual per-IP rate limiting in production, including whether `/v1/positions/*` gets any tighter treatment than an ordinary read given M-2's cost profile.
- Whether `WEB_CONCURRENCY` is set above 1 on the live deployment. If it is, the code's own comments say several in-memory protections (admin-login throttling, the upstream RPC budget) would under-enforce. Not observable from what we reviewed.
- Whether GCP / `api.crackerswap.com` is the only production deployment, or whether something else undocumented exists. Outside what we could check from the repository and the live site. (Separately, the `NODE_ENV=production` question is already resolved, confirmed via the Dockerfile and live testing, so that part isn't in question here.)

High Severity Issues

Seeded admin password gets silently reset on every restart, which undermines credential rotation

Resolved

Path

```
src/admin/auth/admin-seed.service.ts:23, src/database/repositories/admin-user.repository.ts:69
```

Description

AdminSeedService.onApplicationBootstrap runs on every application start, not just the first one, and calls AdminUserRepository.upsertSeeded. That function's update branch unconditionally overwrites the seeded super-admin's password hash from the ADMIN_PASSWORD environment variable. Both production deploy paths (Render, and the GCP docker compose up -d --build path) redeploy on every push to main, so this runs constantly, not just on rare full re-provisions.

Here's why we're calling this High instead of Medium: SECURITY.md says outright that earlier commits leaked a real-looking ADMIN_PASSWORD and treats it as compromised. The obvious fix, changing the password through the /admin/users panel, doesn't actually stick for the seeded account. The next restart quietly reverts it back to whatever ADMIN_PASSWORD holds in the environment. If that variable itself was never rotated (we couldn't verify this, see Section 5), a known-leaked credential could still grant full super-admin access right now, and it'll keep working no matter what gets changed in the database.

Recommendation

make the seed logic create-only so it never overwrites passwordHash on an existing row, or have it skip the password field once a row already exists. And separately, urgently: confirm ADMIN_PASSWORD got rotated as an actual environment variable everywhere it was ever set. A database-only change doesn't count here.

High Severity Issues

Confirmed live: Uniswap aggregator swaps can be redirected to an attacker's address by tampering routeData

Resolved

Path

```
src/adapters/uniswap-api/uniswap-api-swap.adapter.ts, src/adapters/uniswap-api/uniswap-api.route-data.ts
```

Description

We obtained a real session token (the Live Pentest Log covers how) and used it to test whether a swap's payout can be redirected by tampering the data Crackerswap's own /quote response hands back. Here's exactly what we did and what came back:

Fetches a real quote from POST /v1/quote (native ETH into USDC on Base, our own address as sender). The decoded routeData showed our address correctly set as output.recipient and in aggregatedOutputs[0].recipient, the recipient fields an untampered quote carries.

Base64-decoded routeData, changed both of those recipient fields to a different address (0x000...dEaD), left everything else untouched, re-encoded it, and submitted it to POST /v1/swap with a valid Authorization: Bearer header.

Uniswap's Trading API returned real, ready-to-sign transaction calldata. We searched the raw hex: our real address does not appear anywhere in it. The address we substituted appears exactly once, in the position a recipient parameter occupies, immediately following the USDC token address, matching the layout we'd already seen in the untampered response.

In plain terms: Uniswap's Trading API does not validate the quote server-side against its own stored copy. It trusts whatever recipient is in the JSON posted to it and builds a transaction that pays out to that address. Crackerswap's backend passes this straight through with no check of its own.

One detail that changes the fix, not just the severity: the to address on the returned transaction (0x6fF5693b..., Base's Universal Router) is the real, correct router. It isn't hijacked or swapped out, only the recipient parameter inside the calldata is redirected. A simple allow-list check on to would not have caught this (see H-3 below, where the same is true for the LiFi bridge path). A transaction that goes to the legitimate, expected contract, carrying a quietly-redacted payout, is exactly the kind of thing a wallet's "you're interacting with Uniswap" preview would wave through.

Confirmed live: Uniswap aggregator swaps can be redirected to an attacker's address by tampering routeData *(continued)*

Why High. The impact is direct: whatever a swap would have paid out to the real user goes to the attacker's address instead, full stop. What keeps it from being Critical is that exploiting it against a real victim still needs a delivery mechanism, a malicious or compromised frontend, a browser extension, or a MITM sitting between the legitimate /quote and /swap calls. We did not need any of that to demonstrate it against our own test wallet, because we controlled both ends of the request ourselves; a real attack needs to get the tampered request in front of someone else's session.

On the bearer-token gate specifically: it did not protect against this at all, and it's worth being direct about why. The token only gates who can call the endpoint, not whose request gets tampered with. A real victim already has their own valid token from using the app normally. The attack works on their session, not the attacker's, so requiring a token changes nothing about exploitability against a real user. We only needed a token ourselves because we were testing from outside the app, with no session of our own to tamper with.

Recommendation

do not trust the client-supplied quote object when calling Uniswap's /swap endpoint. At minimum, independently verify that the recipient encoded in the returned calldata matches the address the original request actually asked for, before handing anything back to the caller. Better: keep the original quote server-side (keyed by requestId) at the time it's issued, and refuse to execute a /swap request whose posted quote doesn't match byte-for-byte, rather than relying on Uniswap to have already done that check.

High Severity Issues

LiFi bridge execution can be redirected to an attacker's address the same way, and more directly

Resolved

Path

```
src/adapters/lifi/lifi-bridge.adapter.ts:250, src/adapters/lifi/lifi.route-data.ts
```

Description

Same token, same pattern, a bridge instead of a swap, and this one is even more direct than H-2 because there's no third party in the loop at all:

Fetches a real bridge quote from POST /v1/bridge/quote (0.002 WETH, Base to Optimism, our own address on both ends). The decoded routeData contained a quote.transactionRequest.data field, ready-to-sign calldata, with our real address appearing twice inside it.

Directly replaced our address's bytes with a different address's bytes inside that calldata string itself (not just a JSON metadata field, the actual hex), re-encoded routeData, and submitted to POST /v1/bridge/execute with the valid token.

Got back 201 with a real transaction. Our real address appears nowhere in it. The substituted address appears exactly where the destination recipient sits in the encoded call.

The reason this needs no third-party round-trip to fail: per the source, buildBridgeTx's EVM-source path never re-fetches from LiFi at all, it takes quote.transactionRequest straight out of the client-supplied routeData and hands it back unchanged. So this isn't "does LiFi validate a resubmitted quote", the way it was an open question for Uniswap. Crackerswap's own backend is the only thing in the loop, and it does no validation of its own. (Only the Solana-source path in this same adapter re-fetches a fresh quote, so it doesn't have this gap.)

As with H-2, the to address on the returned transaction, LiFi's real router contract, is untouched and correct. Only the recipient encoded inside the calldata is redirected. Same implication for the fix: an allow-list on to alone would not catch this.

Why High, same reasoning as H-2: direct impact (the bridged funds go to the attacker's address instead of the real one), but exploiting it against a real victim still needs a delivery mechanism, a compromised frontend, a malicious extension, or a MITM between quote and execute. The bearer-token gate is irrelevant to that, for the same reason explained in H-2: a real victim already holds their own valid token.

LiFi bridge execution can be redirected to an attacker's address the same way, and more directly *(continued)*

Recommendation

for the EVM-source path specifically, don't return `quote.transactionRequest` untouched. Either re-verify it server-side (compare against what was actually quoted, byte for byte) or, cheaper, independently check that the destination-recipient bytes encoded in the `calldata` match the address the original request named, before handing the transaction back.

Medium Severity Issues

Cost-basis endpoint lets anyone trigger a huge, unauthenticated RPC fan-out

Resolved

Path

```
src/positions/positions.controller.ts:31, src/positions/cost-basis.service.ts:48, src/chain/event-log.service.ts:70
```

Description

GET /positions/:tokenId?includeCostBasis=true takes any tokenId, no ownership check, and on a cache miss scans on-chain event logs across three event types over a per-chain lookback window, in chunks of 100,000 blocks. On Robinhood Chain that window is 45 million blocks, which works out to roughly 1,350 eth_getLogs calls from a single HTTP request.

Position token IDs are sequential and don't require any authentication to query, so an attacker can enumerate a bunch of IDs cheaply (each one a guaranteed cache miss) and fire several of these requests at once. The app's shared RPC budget is deliberately a global bucket rather than per-caller, and it fails open once its wait window is exceeded. So this is a real way for one unauthenticated caller to degrade RPC responsiveness for the whole chain, not just for cost-basis lookups.

Recommendation

add a per-caller or per-IP cap on concurrent cost-basis computations, and/or require some minimum interval between cold computations for new token IDs from the same caller. The generic edge rate limiter has no way to tell this endpoint's cost apart from an ordinary read.

Low Severity Issues

NestJS core has a moderate injection CVE (CWE-74, CVSS 6.1)

Resolved

Path

@nestjs/core and related packages, pinned to ^10.0.0

Description

The advisory needs user interaction and has limited impact, so this is a schedule-it, not a drop-everything.

Recommendation

TBD

Low Severity Issues

multer's high-severity DoS advisories exist but aren't reachable

Resolved

Path

Transitive via @nestjs/platform-express

Description

Checked the whole codebase. Nothing uses FileInterceptor, UploadedFile, or multer anywhere. The vulnerable code only runs if a route calls it, and none do. Worth clearing whenever the NestJS upgrade (L-1) happens anyway.

Low Severity Issues

No explicit HTTP body-size limit configured

Resolved

Path

```
src/main.ts / src/app.module.ts
```

Description

Relies on Express's default (~100KB) instead of a deliberate setting. A future dependency change could quietly alter that default.

Low Severity Issues

No @MaxLength() on any string DTO field

Resolved

Path

Across all DTOs, e.g. routeData, permitSignature, admin token name/symbol

Description

Bounded indirectly by L-3, but an explicit per-field cap is better practice, especially for admin-writable fields that get persisted to the database.

Low Severity Issues

Docker container runs as root

Resolved

Path

`Dockerfile`

Description

The node:20-bookworm-slim base image already ships a built-in node user. Adding USER node before the final CMD is a cheap defense-in-depth win, doesn't fix a specific known exploit but reduces blast radius.

Low Severity Issues

trust proxy hop-count assumption may not match the real network setup

Resolved

Path

```
src/main.ts:30
```

Description

Hardcoded for a single-hop proxy, with a comment assuming Render. Production actually runs behind nginx on a GCP VM. If there's more than one hop, req.ip becomes spoofable via X-Forwarded-For, weakening IP-based admin login lockout. Not observable within this scope.

Low Severity Issues

axios has known advisories, needs a routine version bump

Resolved

Path

package.json, currently resolves to axios 1.16.1; existing range (^1.7.9) already permits the fix

Description

npm audit flagged a proxy-config issue and several prototype-pollution/DoS advisories, fixed in axios 1.18.0+. This is an unpatched-dependency finding, not something we demonstrated as exploited in this app, axios is used behind every third-party integration (CoinGecko, GeckoTerminal, LiFi, Uniswap's Trading API), so it's the most relevant one to clear. Should be a plain lockfile bump and re-test.

Low Severity Issues

qs has known DoS advisories, reachable via public query parameters

Resolved

Path

Transitive via Express's query-string parser (@nestjs/platform-express -> qs)

Description

Same basis as above issue, an unpatched-dependency advisory rather than a confirmed exploit against this app. Reachable in principle from any GET endpoint with query params, which is most of the public API. Fix in the same pass as above issue.

Informational

None of these are defects. They're deliberate tradeoffs, or things that are just inherent to what this kind of API does. Listed here for awareness, not as required fixes.

#	Note	Location	Why it's informational
I-1	ERC-20 approvals are always unlimited (MaxUint256), never the exact swap amount	src/chain/approval.service.ts:41	Deliberate, standard DeFi UX, matches Uniswap's own interface, avoids repeated approval prompts. Does widen the blast radius if a router contract is ever compromised. Worth knowing, not worth changing without a product conversation.

#	Note	Location	Why it's informational
I-2	recipient/userAddress fields are self-declared with no signature binding	Every swap/liquidity endpoint	By design. The real authorization boundary is on-chain (wallet signing, NFT ownership checks). A malicious frontend could build a transaction with an attacker-controlled recipient and get a victim's wallet to sign it, but that's true of any unauthenticated, stateless transaction-builder API, including Uniswap's own. Not fixable in this codebase. The mitigation has to live in wallet-side calldata transparency.

#	Note	Location	Why it's informational
I-3	Token name/symbol/logoURI are unsanitized, on-chain-sourced free text	src/tokens/verified-token-list.service.ts:335	Anyone can deploy an ERC-20 with an arbitrary name/symbol and get it indexed through a permissionless pool; this backend stores and returns it as-is, which is correct behavior for a DEX API. We checked whether this could become stored XSS by inspecting the live frontend bundle: token/chain icons render through <code></code> or SVG <code><image href></code> (both safe against script execution) and name/symbol go through standard auto-escaping JSX text, with no <code>dangerouslySetInnerHTML</code> /.innerHTML anywhere near token data. Not exploitable on the frontend as it exists today. Still a trust-boundary note for the frontend team going forward, and an attacker-controlled logoURI can still leak viewer IPs or spoof a convincing scam-token icon regardless of XSS.

#	Note	Location	Why it's informational
I-4	An unsupported or unconfigured chainId returns a generic 500 instead of a clean 400	Chain/protocol dispatch layer	Its Fixed by crackerswap team,Just a UX nit. The global exception filter already guarantees nothing internal leaks in production, so this is about a clearer error message, not security.

Closing Summary

In this report, we have considered the security of the Crackerswap Backend Code and Live URL. We performed our audit according to the procedure described above.

Some issues of High, medium and low severity were found; some suggestions and best practices are also provided to improve the code quality and security posture.

Disclaimer

QuillAudits Dapp security audit provides services to help identify and mitigate potential security risks in the Crackerswap. However, it is important to understand that no security audit can guarantee complete protection against all possible security threats. QuillAudits audit reports are based on the information provided to us at the time of the audit, and we cannot guarantee the accuracy or completeness of this information. Additionally, the security landscape is constantly evolving, and new security threats may emerge after the audit has been completed.

Therefore, it is recommended that multiple audits and bug bounty programs be conducted to ensure the ongoing security of Crackerswap. One audit is not enough to guarantee complete protection against all possible security threats. It is important to implement proper risk management strategies and stay vigilant in monitoring your smart contracts for potential security risks.

QuillAudits cannot be held liable for any security breaches or losses that may occur subsequent to and despite using our audit services. It is the responsibility of the Crackerswap to implement the recommendations provided in our audit reports and to take appropriate steps to mitigate potential security risks.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With seven years of expertise, we've secured over 1400 projects globally, averting over \$3 billion in losses.

Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.



8+ Years of Expertise	1M+ Lines of Code Audited
50+ Chains Supported	1500+ Projects Secured

Follow Our Journey



AUDIT REPORT

September 2026

For



Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com